

TRILHA TÉCNICA COMPLETA

Inteligência Artificial

Do básico ao avançado/sênior

PDF técnico profissional para estudo, consulta e evolução progressiva em IA, Machine Learning, Deep Learning, LLMs, IA Generativa, RAG, MLOps, arquitetura, segurança e governança.

Formato	Trilha de estudos técnica e editorial
Progressão	Fundamentos → Intermediário → Avançado → Profissional/Sênior
Uso	Estudo guiado, consulta recorrente, portfólio técnico e planejamento profissional

Documento gerado a partir do conteúdo anexado, com preservação do conteúdo técnico e reorganização editorial para leitura profissional.

Sumário

Objetivo do material	8
1. Introdução	9
2. Visão geral da trilha	9
3. Como usar este guia	9
4. Ferramentas recomendadas	10
5. Visão geral da área de Inteligência Artificial	11
IA	11
Machine Learning	11
Deep Learning	11
Data Science	11
IA Generativa	11
LLMs	11
6. História e evolução da IA	11
7. Mercado e papéis profissionais	11
Data Scientist	12
Machine Learning Engineer	12
AI Engineer / LLM Engineer	12
MLOps Engineer / Platform Engineer	12
AI Architect	12
8. Mapa mental dos principais domínios da IA	12
9. Fundamentos obrigatórios	15
Conceitos centrais	15
Aprendizado não supervisionado	15
Aprendizado por reforço	15
Aprendizado auto-supervisionado	15
Tipos de problema	15
Aplicações de mercado	16
Limitações e riscos	16
Erros comuns	16
Critério de domínio	17
10. Matemática para IA	17
Álgebra linear	17
Conceitos principais	17
Para que serve em IA	17
Cálculo diferencial	17
Conceitos principais	17
Probabilidade	18
Conceitos principais	18
Estatística	18
Conceitos principais	18
Otimização	18

Conceitos principais	18
O que estudar primeiro	19
Práticas recomendadas	19
Ferramentas	19
Erros comuns	19
Critério de domínio	19
11. Programação para IA	19
Estruturas de dados	20
NumPy	20
Pandas	20
Matplotlib e Seaborn	20
Scikit-learn	21
Jupyter Notebook	21
Git	21
Organização de projeto	21
Boas práticas de engenharia	21
Ferramentas	22
Erros comuns	22
Critério de domínio	22
12. Dados e engenharia de dados para IA	22
Limpeza	22
Dados ausentes	23
Normalização	23
Padronização	23
Encoding	23
Feature engineering	23
Feature selection	23
Train/validation/test	23
Data leakage	23
SQL e NoSQL	24
Data pipelines	24
Governança de dados	24
Erros comuns	24
Critério de domínio	24
13. Machine Learning	25
Aprendizado supervisionado	25
Algoritmos principais	25
Métricas	25
Aprendizado não supervisionado	25
Algoritmos principais	25
Métricas	25
Aprendizado por reforço	26
Underfitting	26
Bias	26

Variance	26
Validação cruzada	26
Tuning de hiperparâmetros	26
Baseline	26
Curva de aprendizado	26
Verdade prática de mercado	26
Ferramentas	26
Erros comuns	27
Critério de domínio	27
14. Deep Learning	27
Perceptron	27
MLP	27
Funções de ativação	27
Backpropagation	27
Otimizadores	27
CNN	27
RNN, LSTM e GRU	28
Autoencoders	28
Transformers	28
Embeddings	28
Attention	28
Transfer learning	28
Fine-tuning	28
Regularização	28
PyTorch vs TensorFlow/Keras	29
Erros comuns	29
Critério de domínio	29
15. Processamento de Linguagem Natural — NLP	29
Stemming	29
Lematização	29
Stopwords	29
TF-IDF	29
Embeddings clássicos	29
Modelos contextuais	30
Tarefas de NLP	30
NLP moderno	30
Aplicações reais	30
Erros comuns	30
Critério de domínio	31
16. IA Generativa e LLMs	31
LLMs	31
Tokens	31
Context window	31
Embeddings	31

Pre-training	31
Fine-tuning	31
Instruction tuning	31
RLHF	31
Prompt engineering	31
System prompt	31
Few-shot prompting	31
Raciocínio estruturado em produção	31
RAG — Retrieval-Augmented Generation	32
Componentes de um RAG	32
Bancos vetoriais	32
Agentes	32
Tool use e function calling	32
Guardrails	32
Avaliação de LLMs	33
Erros comuns	33
Critério de domínio	33
17. Engenharia de aplicações com IA	34
Arquitetura típica de uma aplicação com LLM	34
Modelos locais ou self-hosted	34
Confiabilidade e custo	35
Observabilidade	35
Segurança e privacidade	35
Exemplo de arquitetura para portfólio	36
Critério de domínio	36
18. MLOps e ciclo de vida de modelos	36
Ciclo de vida de modelos	36
Experiment tracking	36
Dados	37
Modelos	37
Feature Store	37
Deploy e serving	37
Monitoramento	38
Concept drift	38
Model drift	38
Retreino	38
CI/CD para ML	39
Erros comuns	39
Critério de domínio	39
19. Cloud e infraestrutura para IA	39
Componentes de infraestrutura	39
AWS	40
Azure	40
Google Cloud	40

Serving de LLMs	40
Estratégias de deploy	40
Trade-offs	41
Erros comuns	41
Critério de domínio	41
20. Segurança, ética e governança em IA	41
Data exfiltration	41
Insecure output handling	41
Model denial of service	41
Supply chain	41
OWASP LLM Top 10	42
Privacidade	42
Ética	42
Governança	42
NIST AI RMF	42
Erros comuns	43
Critério de domínio	43
21. Projetos práticos por nível	44
Nível 1 — Fundamentos	44
Nível 2 — Intermediário	45
Nível 3 — Avançado	46
Nível 4 — Profissional/Sênior	49
22. Grade de estudos do básico ao avançado	51
Nível 1 — Fundamentos	51
Nível 2 — Intermediário	52
Nível 3 — Avançado	52
Nível 4 — Profissional/Sênior	53
23. Cronograma de 3 meses — imersivo e aplicado	54
24. Cronograma de 6 meses — base sólida	54
25. Cronograma de 12 meses — formação avançada	55
26. Cronograma de 18 meses — nível profissional/sênior	55
Projeto final de 18 meses	55
27. Fontes oficiais e referências	56
Materiais de universidades	56
Bibliotecas e frameworks	56
LLMs, RAG e bancos vetoriais	56
MLOps	56
Cloud	56
Segurança, privacidade e governança	56
Outras documentações técnicas	56
Livros recomendados	56
28. Checklist de evolução técnica	57
Fundamentos	57

Matemática	57
Programação e dados	57
Machine Learning	57
Deep Learning e NLP	57
LLMs e IA Generativa	58
Engenharia e MLOps	58
Cloud e infraestrutura	58
Segurança e governança	58
Sênior	58
29. Resultado esperado ao final da trilha	60
30. Conclusão editorial	60

Objetivo do material

Objetivo editorial

Organizar uma pesquisa técnica completa sobre Inteligência Artificial como uma trilha de estudos progressiva, adequada para estudo, consulta, planejamento profissional e evolução do nível iniciante ao profissional/sênior.

A proposta não é apenas aprender ferramentas ou usar modelos prontos. O objetivo é desenvolver competência real para entender, construir, integrar, avaliar, publicar, monitorar, proteger e governar sistemas inteligentes em produção.

A trilha cobre:

Fundamentos de IA, matemática, programação, dados, Machine Learning, Deep Learning, NLP, LLMs, IA Generativa, RAG, agentes, engenharia de aplicações com IA, MLOps, cloud, infraestrutura, segurança, ética, governança, projetos práticos, cronogramas e checklist de evolução técnica.

1. Introdução

Inteligência Artificial deixou de ser apenas um campo acadêmico ou experimental. Hoje, IA é uma disciplina de engenharia aplicada que atravessa praticamente todo o desenvolvimento de software moderno.

Para quem já vem de backend, Java, Spring, Python, APIs, banco de dados, cloud ou sistemas distribuídos, a transição mais saudável não é “trocar de área”, mas adicionar uma nova camada de competência técnica: continuar entregando sistemas, mas agora com capacidade de embutir inferência, recuperação semântica, automação inteligente, modelos preditivos e aplicações generativas.

A sequência central de aprendizagem é:

Fundamentos → Matemática → Python e dados → Machine Learning clássico → Deep Learning → NLP e LLMs → Engenharia de aplicações → MLOps → Cloud → Segurança → Governança → Projetos profissionais.

A ideia central da trilha é simples: você só domina IA quando consegue levar uma solução do estudo ou notebook até produção, com avaliação, monitoramento, custo controlado, segurança, rastreabilidade e valor real para o negócio.

Treinar um modelo é apenas uma parte do trabalho. Em projetos reais, grande parte do esforço está em dados, arquitetura, validação, deploy, observabilidade, manutenção, retreinamento, segurança e governança.

2. Visão geral da trilha

A visão geral foi organizada como uma matriz de progressão técnica, preservando os níveis, focos e resultados esperados do conteúdo original.

Nível	Foco técnico	Resultado esperado
Nível 1 — Fundamentos	Conceitos básicos, história da IA, tipos de problema, matemática inicial, Python, dados básicos e primeiros modelos.	Entender o que é IA, diferenciar IA, Machine Learning, Deep Learning, IA Generativa e LLMs, preparar datasets simples e treinar os primeiros modelos.
Nível 2 — Intermediário	Machine Learning clássico, avaliação de modelos, feature engineering, validação cruzada, tuning, pipelines e projetos com datasets reais.	Construir e comparar modelos supervisionados e não supervisionados com validação correta, evitando erros como data leakage, overfitting e escolha inadequada de métricas.
Nível 3 — Avançado	Deep Learning, NLP, Transformers, embeddings, IA Generativa, LLMs, RAG, bancos vetoriais, fine-tuning, agentes e aplicações inteligentes.	Criar aplicações modernas com modelos de linguagem, busca semântica, recuperação aumentada por geração, classificação de texto/imagem e integração com ferramentas externas.
Nível 4 — Profissional/Sênior	Arquitetura, MLOps, cloud, segurança, governança, observabilidade, escalabilidade, custo, deploy e operação de sistemas de IA em produção.	Projetar, operar e governar soluções reais de IA, com APIs, pipelines, monitoramento, segurança, LGPD, rastreabilidade, auditoria, controle de custo e decisões arquiteturais justificáveis.

3. Como usar este guia

Leia os capítulos na ordem proposta. Pratique cada bloco com pequenos laboratórios. Só avance para projetos mais complexos quando conseguir explicar os conceitos, reproduzir os experimentos, justificar decisões técnicas e identificar erros comuns.

A trilha deve ser usada em três camadas:

- Primeira camada: leitura e entendimento conceitual.

Segunda camada: prática com código, dados e experimentos.

Terceira camada: construção de projetos publicáveis em portfólio.

O objetivo não é terminar rápido. O objetivo é construir domínio progressivo.

4. Ferramentas recomendadas

Categoria	Ferramentas
Programação e dados	Python, Jupyter Notebook, VS Code, NumPy, Pandas, Matplotlib, Seaborn, Git.
Machine Learning	Scikit-learn, XGBoost, LightGBM, CatBoost, MLflow.
Deep Learning	PyTorch, TensorFlow/Keras.
NLP e LLMs	Hugging Face Transformers, OpenAI API, Anthropic, Google Gemini, LangChain, LlamaIndex.
RAG e bancos vetoriais	Pinecone, Weaviate, Qdrant, Milvus, Chroma, pgvector.
MLOps	MLflow, DVC, Feast, Kubeflow, Airflow, Docker, Kubernetes, Evidently.
Cloud e infraestrutura	AWS SageMaker AI, Azure AI Foundry, Google Vertex AI, GPUs, TPUs, vLLM, Ollama.
Segurança e governança	OWASP LLM Top 10, NIST AI RMF, LGPD, GDPR, políticas internas de auditoria, model cards, data cards.

5. Visão geral da área de Inteligência Artificial

Inteligência Artificial é o campo da computação que cria sistemas capazes de executar tarefas associadas à inteligência humana, como aprender padrões, reconhecer imagens, entender linguagem, raciocinar, tomar decisões, gerar conteúdo e interagir com ambientes.

IA é um guarda-chuva. Dentro dele existem várias camadas:

IA

- Sistemas inteligentes em sentido amplo.

Exemplos: assistentes virtuais, recomendadores, automação, agentes inteligentes.

Machine Learning

Modelos que aprendem padrões a partir de dados em vez de seguir apenas regras programadas manualmente.

Exemplos: previsão de churn, score de crédito, detecção de fraude, classificação de documentos.

Deep Learning

Subárea de Machine Learning baseada em redes neurais profundas.

Exemplos: visão computacional, reconhecimento de voz, NLP, modelos generativos.

Data Science

Disciplina focada em extrair valor analítico de dados usando estatística, visualização, experimentação, análise exploratória e Machine Learning.

IA Generativa

Classe de modelos capazes de gerar novos conteúdos, como texto, imagem, áudio, vídeo e código.

LLMs

Large Language Models, ou modelos grandes de linguagem, são modelos generativos treinados em grandes volumes de texto para prever, completar, transformar e gerar linguagem.

6. História e evolução da IA

A evolução da IA pode ser entendida como uma sequência de ondas técnicas:

- 1950 — Teste de Turing.

1956 — Conferência de Dartmouth, onde o termo Inteligência Artificial se popularizou.

Décadas seguintes — Sistemas simbólicos baseados em regras.

Períodos de inverno da IA — Fases de frustração por expectativas exageradas e limitações técnicas.

- 1986 — Popularização do backpropagation.

2012 — AlexNet mostra o poder de CNNs com GPUs, marcando o avanço do Deep Learning moderno.

2017 — O paper “Attention Is All You Need” apresenta a arquitetura Transformer.

2018 a 2020 — Avanço de BERT, GPT e modelos pré-treinados.

2022 em diante — Explosão de LLMs, RAG, agentes, multimodalidade e IA Generativa aplicada.

A lição central é que avanços em IA não vêm de um único fator isolado. Eles surgem da combinação entre algoritmos, dados, capacidade computacional, infraestrutura e aplicação correta.

7. Mercado e papéis profissionais

A nomenclatura de cargos ajuda a orientar o estudo.

Data Scientist

Foca em estatística, experimentação, análise de dados, modelagem, hipóteses, visualização e comunicação de insights.

Machine Learning Engineer

Leva modelos para produção, constrói pipelines, automatiza treino, deploy, validação, monitoramento e integração com sistemas.

AI Engineer / LLM Engineer

Constrói aplicações sobre modelos de fundação, como RAG, agentes, copilotos, chatbots, orquestração de ferramentas e integração de LLMs com sistemas reais.

MLOps Engineer / Platform Engineer

Mantém infraestrutura, versionamento, observabilidade, CI/CD de ML, monitoramento de modelos, drift, registry, deploy e escalabilidade.

AI Architect

Desenha arquitetura ponta a ponta, define trade-offs entre modelo local/provedor externo, cloud, custo, latência, segurança, governança e integração corporativa.

Para um perfil de software com experiência em backend, APIs, Java/Spring, Python, banco de dados e cloud, os caminhos mais naturais são AI Engineer, LLM Engineer, ML Engineer e, futuramente, AI Architect.

8. Mapa mental dos principais domínios da IA

Mapa dos domínios principais da trilha. Cada bloco agrupa os conceitos preservados do conteúdo original.

Fundamentos

- História.
- Tipos de IA.
- Aplicações.
- Limitações.
- Riscos.
- Diferenças entre IA, ML, Deep Learning, GenAI e LLMs.

Matemática

- Álgebra linear.
- Vetores.
- Matrizes.
- Tensores.
- Cálculo.
- Derivadas.
- Gradientes.
- Probabilidade.
- Estatística.
- Otimização.
- Funções de perda.
- Métricas.

Programação

- Python.
- NumPy.
- Pandas.
- Matplotlib.
- Scikit-learn.
- PyTorch.
- TensorFlow/Keras.
- Git.
- Jupyter.
- Organização de projetos.

Dados

- Coleta.
- Limpeza.
- Tratamento de dados ausentes.
- Normalização.
- Padronização.
- Encoding.
- Feature engineering.
- Feature selection.
- Pipelines.
- SQL.
- NoSQL.
- Data lakes.
- Governança.

Machine Learning

- Aprendizado supervisionado.
- Aprendizado não supervisionado.
- Aprendizado por reforço.
- Validação.
- Métricas.
- Overfitting.
- Underfitting.
- Bias.
- Variance.
- Tuning.
- Baselines.

Deep Learning

- Perceptron.
- MLP.
- CNN.
- RNN.
- LSTM.
- GRU.
- Autoencoders.
- Transformers.
- Embeddings.
- Attention.
- Transfer learning.
- Fine-tuning.

NLP

- Tokenização.
- Stemming.
- Lematização.
- Stopwords.
- Bag of Words.
- TF-IDF.
- Word2Vec.
- GloVe.
- FastText.
- BERT.
- RoBERTa.
- DistilBERT.
- Classificação.
- NER.
- QA.
- Busca semântica.

IA Generativa e LLMs

- Pre-training.
- Fine-tuning.
- Instruction tuning.
- RLHF.
- Prompt engineering.
- System prompt.
- Few-shot prompting.
- Embeddings.
- Context window.
- RAG.
- Agentes.
- Tool use.
- Function calling.
- Guardrails.
- Avaliação.

Engenharia de Aplicações com IA

- APIs.
- Backend.
- RAG em produção.
- Streaming.
- Cache.
- Filas.
- Observabilidade.
- Rate limits.
- Fallback.
- Logs.
- Métricas de custo.
- Segurança.

MLOps

- Experimentos.
- Tracking.
- Versionamento de dados.
- Versionamento de modelos.
- Model registry.
- Deploy.
- Serving.
- Monitoramento.
- Drift.
- Retreino.
- CI/CD.
- Governança operacional.

Cloud e Infraestrutura

- AWS SageMaker.
- Azure AI Foundry.
- Google Vertex AI.
- GPUs.
- TPUs.
- Kubernetes.
- Docker.
- vLLM.
- Ollama.
- Bancos vetoriais.
- Object storage.
- Data lakes.

Segurança, Ética e Governança

- LGPD.
- GDPR.
- NIST AI RMF.
- OWASP LLM Top 10.
- Fairness.
- Viés.
- Explicabilidade.
- Auditoria.
- Compliance.
- Prompt injection.
- Data poisoning.
- Model inversion.
- Rastreabilidade.

9. Fundamentos obrigatórios

O primeiro bloco da trilha deve responder perguntas essenciais:

O que é IA?

Como Machine Learning se diferencia de programação tradicional?

Por que dados são essenciais?

Quando usar regras determinísticas?

Quando usar modelos?

Quando usar LLM?

Quando usar RAG?

Quando usar fine-tuning?

Quando usar busca tradicional?

- Quais problemas IA resolve bem?
- Quais problemas IA não resolve bem?

Conceitos centrais

- IA é o campo mais amplo.
- Machine Learning é uma subárea da IA.

Deep Learning é uma subárea de Machine Learning.

IA Generativa geralmente usa Deep Learning para gerar novos conteúdos.

LLMs são modelos generativos de linguagem em larga escala.

Data Science se sobrepõe parcialmente a ML, mas também envolve estatística, visualização, análise e experimentação.

- Tipos de aprendizado
- Aprendizado supervisionado
- Usado quando existe rótulo ou valor-alvo.

Exemplos: classificação de spam, previsão de preço, previsão de churn.

Aprendizado não supervisionado

- Usado quando não há rótulo.

Exemplos: clustering, segmentação de clientes, redução de dimensionalidade.

Aprendizado por reforço

Usado quando um agente aprende por tentativa, erro e recompensa.

Exemplos: jogos, robótica, controle, otimização de decisões e alguns processos de alinhamento de modelos.

Aprendizado auto-supervisionado

Usado quando o próprio dado fornece o sinal de treinamento.

É uma base importante para modelos modernos de linguagem.

Tipos de problema

- Classificação.
- Regressão.
- Clustering.
- Ranking.
- Recomendação.
- Geração.
- Sumarização.

- Busca semântica.
- Detecção de anomalia.
- Extração de entidades.
- Question answering.
- Previsão de séries temporais.

Aplicações de mercado

- Busca semântica.
- Assistentes internos.
- Copilotos de código.
- Detecção de fraude.
- Previsão de demanda.
- Análise de churn.
- Classificação de documentos.
- Análise de sentimento.
- Atendimento automatizado.
- Geração de documentos.
- Análise de contratos.
- Visão computacional.
- OCR.
- Manutenção preditiva.
- Educação.
- Finanças.
- Saúde.
- Segurança.
- Automação corporativa.

Limitações e riscos

- Modelos dependem de dados de qualidade.
- Modelos podem refletir vieses dos dados.
- LLMs podem alucinar.
- Métricas podem enganar.
- Sistemas podem degradar com drift.
- Custos podem sair do controle.
- Explicabilidade pode ser limitada.
- Privacidade deve ser tratada desde o início.
- Prompt injection pode comprometer aplicações com LLM.
- Modelos bons em notebook não são automaticamente bons em produção.

Erros comuns

- Achar que IA é sempre LLM.
- Usar modelo antes de entender o problema.
- Ignorar qualidade dos dados.
- Confundir acurácia com qualidade real.
- Ignorar custo de inferência.
- Não avaliar alucinação.
- Não validar em produção.
- Não pensar em segurança desde o início.

Critério de domínio

Você domina os fundamentos quando consegue explicar, com exemplos reais, quando usar regressão, classificação, clustering, RAG, fine-tuning, regras determinísticas, busca lexical, busca semântica e LLMs.

10. Matemática para IA

Você não precisa estudar matemática como uma graduação completa no início, mas precisa de intuição operacional. A regra prática é aprender matemática junto com o código que a utiliza.

A matemática serve para entender como modelos representam dados, medem erro, ajustam parâmetros, aprendem padrões e avaliam resultados.

Álgebra linear

Álgebra linear é a linguagem dos modelos.

Conceitos principais

- Vetores.
- Matrizes.
- Tensores.
- Produto escalar.
- Multiplicação de matrizes.
- Norma de vetores.
- Espaço vetorial.
- Similaridade.
- Distância.
- Embeddings.

Para que serve em IA

- Representar dados.
- Representar textos como embeddings.
- Representar imagens como tensores.
- Calcular similaridade semântica.
- Operar camadas de redes neurais.
- Entender atenção em Transformers.
- Entender bancos vetoriais.

Uma imagem pode ser vista como um tensor.

- Um embedding de texto é um vetor.
- Um lote de exemplos é uma matriz.

Uma camada de rede neural envolve multiplicação de matrizes e funções não lineares.

Cálculo diferencial

Cálculo ajuda a entender como modelos aprendem.

Conceitos principais

- Derivada.
- Gradiente.
- Derivadas parciais.
- Regra da cadeia.
- Backpropagation.
- Taxa de variação.

- Learning rate.

O treinamento de redes neurais é, essencialmente, um processo de otimização via gradientes. Frameworks como PyTorch e TensorFlow calculam gradientes automaticamente, mas você precisa entender o que está acontecendo para diagnosticar erros de treinamento.

Probabilidade

Probabilidade permite lidar com incerteza.

Conceitos principais

- Eventos.
- Probabilidade condicional.
- Teorema de Bayes.
- Distribuições.
- Normal.
- Binomial.
- Poisson.
- Incerteza.
- Classificadores probabilísticos.

Estatística

Estatística é essencial para interpretar dados e resultados.

Conceitos principais

- Média.
- Mediana.
- Moda.
- Variância.
- Desvio-padrão.
- Correlação.
- Intervalos de confiança.
- Testes de hipótese.
- P-valor.
- Amostragem.
- Distribuições.
- Análise exploratória.

Sem estatística, fica difícil saber se uma melhora de métrica é real ou apenas ruído.

Otimização

Otimização é o processo de ajustar parâmetros para reduzir erro.

Conceitos principais

- Função de perda.
- MSE.
- MAE.
- Cross-entropy.
- Gradiente descendente.
- SGD.
- Adam.
- AdamW.
- Convergência.
- Mínimos locais.

- Learning rate.

A função de perda traduz o erro do modelo em um número. O otimizador ajusta os pesos para reduzir esse número.

O que estudar primeiro

- Vetores.
- Matrizes.
- Média.
- Variância.
- Desvio-padrão.
- Probabilidade básica.
- Derivadas.
- Função de perda.
- Métricas.
- Produto escalar.
- Multiplicação de matrizes.

Práticas recomendadas

- Implementar produto escalar em NumPy.
- Implementar multiplicação de matrizes.
- Visualizar gradiente descendente com Matplotlib.
- Codar regressão linear do zero com NumPy.
- Calcular métricas estatísticas manualmente e comparar com Pandas.
- Criar tensores em PyTorch.
- Simular diferentes learning rates.
- Visualizar distribuições.

Ferramentas

- NumPy.
- SciPy.
- Matplotlib.
- Jupyter Notebook.
- PyTorch.

Erros comuns

- Decorar fórmulas sem intuição.
- Estudar matemática isolada sem código.
- Tentar entender Transformers sem dominar multiplicação de matrizes.
- Ignorar normalização.
- Não entender função de perda.
- Confundir loss com métrica de negócio.

Critério de domínio

Você domina essa etapa quando consegue explicar por que um modelo reduz erro, como uma função de perda orienta treinamento, por que vetores representam dados e por que a atenção usa produto escalar.

11. Programação para IA

Python é a linguagem dominante para IA por causa do ecossistema de bibliotecas. Aplicações reais podem usar backend em Java, Node, Go ou Python consumindo modelos via API, mas Python é obrigatório para dados, modelagem e experimentação.

Para quem já programa, o foco não é aprender lógica do zero. O foco é fluência em Python científico, organização de projetos, reprodutibilidade e boas práticas.

- Conteúdos essenciais
- Python
- Funções.
- Classes.
- Módulos.
- Typing.
- Ambientes virtuais.
- Manipulação de arquivos.
- Tratamento de erros.
- Organização de pacotes.

Objetivo: escrever código limpo, reutilizável e testável.

Estruturas de dados

- Listas.
- Dicionários.
- Tuplas.
- Sets.
- Compreensões.
- Iteradores.

Objetivo: manipular datasets, pipelines e estruturas de transformação.

NumPy

- Arrays.
- Broadcasting.
- Álgebra linear.
- Operações vetorizadas.
- Geração aleatória.
- Tensores.

Objetivo: computação numérica eficiente.

Pandas

- DataFrames.
- Series.
- Joins.
- GroupBy.
- Missing values.
- Tipos de dados.
- Filtros.
- Agregações.
- Leitura e escrita de CSV, Excel, Parquet.

Objetivo: manipulação tabular e preparação de datasets.

Matplotlib e Seaborn

- Gráficos.
- Distribuições.
- Séries temporais.
- Histogramas.
- Boxplots.

- Scatter plots.
- Visualização de métricas.

Objetivo: análise visual e comunicação de resultados.

Scikit-learn

- Pipelines.
- Modelos clássicos.
- Métricas.
- Validação cruzada.
- Model selection.
- Tuning.
- Pré-processamento.

Objetivo: Machine Learning clássico.

Jupyter Notebook

- Exploração.
- Documentação.
- Experimentos.
- Análise interativa.

Objetivo: prototipação e comunicação técnica.

Git

- Commits.
- Branches.
- Tags.
- Versionamento.
- Colaboração.
- Reprodutibilidade.

Objetivo: controlar evolução do projeto.

Organização de projeto

Estrutura recomendada:

- src/
- data/
- notebooks/
- tests/
- configs/
- models/
- reports/
- requirements.txt ou pyproject.toml
- README.md

Boas práticas de engenharia

- Separar exploração de produção.
- Usar ambientes virtuais.
- Fixar versões de bibliotecas.
- Criar scripts reproduzíveis.
- Evitar notebooks gigantes e desorganizados.
- Usar seeds fixas.
- Criar funções puras e testáveis.
- Escrever testes para pré-processamento.

- Usar type hints.
- Versionar experimentos.
- Documentar decisões.
- Evitar loops Python quando NumPy vetorizado resolve melhor.

Ferramentas

- Python.
- Jupyter.
- VS Code.
- NumPy.
- Pandas.
- Matplotlib.
- Scikit-learn.
- PyTorch.
- Git.
- pytest.
- Poetry, uv, conda ou venv.

Erros comuns

- Misturar exploração e produção no mesmo notebook.
- Não fixar versões.
- Não versionar dados ou modelos.
- Escrever código impossível de reproduzir.
- Não separar funções reutilizáveis.
- Usar loops ineficientes.
- Não criar README.
- Não salvar pipeline de pré-processamento.

Critério de domínio

Você domina programação para IA quando consegue pegar um CSV real, limpar dados, criar features, treinar um modelo, avaliar métricas, salvar artefatos e organizar tudo em um repositório Git reproduzível.

12. Dados e engenharia de dados para IA

IA depende diretamente de dados. Em sistemas reais, a maior parte do esforço não está no algoritmo, mas em coleta, limpeza, transformação, versionamento, governança e monitoramento de dados.

O ditado “garbage in, garbage out” é literal: dados ruins produzem modelos ruins.

- Ciclo de tratamento de dados
- Coleta

Extração via APIs, bancos relacionais, arquivos, eventos, logs, scraping, data lakes e streams.

Cuidados: proveniência, licença, privacidade, qualidade, frequência de atualização e finalidade.

Limpeza

- Correção de inconsistências.
- Remoção de duplicatas.
- Correção de tipos.
- Tratamento de outliers.
- Validação de valores impossíveis.
- Padronização de formatos.

Dados ausentes

- Remoção.
- Imputação por média.
- Imputação por mediana.
- Imputação por modelo.
- Criação de flag indicando ausência.
- Tratamento por domínio.

A escolha errada pode destruir sinal ou gerar viés.

Normalização

Escala valores para intervalo comum, como 0 a 1.

Útil para KNN, SVM, redes neurais e algoritmos sensíveis à escala.

Padronização

Transforma dados para média 0 e desvio-padrão 1.

Útil em modelos que dependem de distância ou gradiente.

Encoding

- One-hot encoding.
- Label encoding.
- Ordinal encoding.
- Target encoding.
- Embeddings.

Usado para transformar variáveis categóricas em representação numérica.

Feature engineering

Criação de variáveis úteis a partir dos dados.

Exemplos:

- Total gasto nos últimos 30 dias.
- Tempo desde última compra.
- Média de acessos por semana.
- Quantidade de reclamações.
- Razão entre variáveis.
- Flags de comportamento.

Feature selection

Remoção de variáveis irrelevantes, redundantes ou ruidosas.

Reduz custo, melhora interpretabilidade e pode melhorar generalização.

Train/validation/test

Separação em treino, validação e teste.

- Treino: usado para ajustar o modelo.
- Validação: usado para escolher hiperparâmetros.
- Teste: usado apenas no final para estimar generalização.

Data leakage

Data leakage é vazamento de informação do futuro ou do conjunto de teste para o treino.

Exemplos:

- Normalizar usando estatísticas do dataset inteiro antes da divisão.

- Usar variável que só existe depois do evento previsto.
- Fazer feature engineering usando dados futuros.
- Escolher hiperparâmetros olhando o teste.
- Avaliar no mesmo conjunto usado para treinar.

Sinal de alerta: acurácia de 99% em problema difícil geralmente indica vazamento, não genialidade.

SQL e NoSQL

SQL é essencial para dados estruturados, joins, agregações e datasets analíticos.

NoSQL é útil para documentos, eventos, logs, dados semiestruturados e armazenamento flexível.

Em IA Generativa, bancos vetoriais entram para armazenar embeddings e permitir busca semântica.

Data pipelines

Pipelines automatizam o fluxo:

Coleta → Transformação → Validação → Feature engineering → Armazenamento → Treino → Deploy → Monitoramento.

Ferramentas possíveis:

- Airflow.
- dbt.
- Dagster.
- Prefect.
- Scripts orquestrados.
- Kafka.
- Spark.
- Flink.

Governança de dados

- Qualidade.
- Catálogo.
- Linhagem.
- Controle de acesso.
- Rastreabilidade.
- Conformidade.
- LGPD.
- Auditoria.
- Documentação.

Erros comuns

- Treinar com dados vazados.
- Avaliar no mesmo conjunto de treino.
- Não salvar pipeline de pré-processamento.
- Limpar treino e produção de formas diferentes.
- Ignorar distribuição dos dados.
- Não documentar origem dos dados.
- Não validar qualidade.
- Não pensar em privacidade.

Critério de domínio

Você domina dados para IA quando consegue construir um pipeline reproduzível que transforma dados brutos em features treináveis, aplicáveis em produção, versionadas e auditáveis.

13. Machine Learning

Machine Learning é o núcleo da IA preditiva. É onde se aprendem conceitos transferíveis para quase tudo: overfitting, underfitting, validação, métricas, vies, variância, regularização e generalização.

Antes de pular para LLMs, é essencial dominar Machine Learning clássico.

Aprendizado supervisionado

Usado quando há dados com rótulo ou valor-alvo.

Algoritmos principais

- Regressão linear: previsão de valor contínuo, baseline, interpretabilidade.
- Regressão logística: classificação binária.
- Decision Tree: regras interpretáveis.
- Random Forest: ensemble robusto.
- Gradient Boosting: alto desempenho em dados tabulares.
- XGBoost, LightGBM, CatBoost: modelos fortes para produção tabular.
- SVM: margens e separação.
- KNN: similaridade local.
- Naive Bayes: texto e classificação probabilística simples.

Métricas

Para regressão:

- MAE.
- MSE.
- RMSE.
- MAPE.
- R^2 .

Para classificação:

- Accuracy.
- Precision.
- Recall.
- F1.
- ROC-AUC.
- PR-AUC.
- Matriz de confusão.

Aprendizado não supervisionado

Usado quando não há rótulos.

Algoritmos principais

- K-Means: segmentação.
- DBSCAN: clusters por densidade.
- PCA: redução de dimensionalidade.
- Gaussian Mixture: clusters probabilísticos.
- Isolation Forest: detecção de anomalias.

Métricas

- Silhouette score.
- Davies-Bouldin.
- Inércia.

- Análise qualitativa.

Aprendizado por reforço

Usado quando um agente aprende por tentativa, erro e recompensa.

Aplicações:

- Jogos.
- Robótica.
- Controle.
- Otimização de decisões.
- Alinhamento de modelos.
- RLHF.
- Conceitos críticos
- Overfitting

O modelo decora o treino e falha em dados novos.

Underfitting

O modelo é simples demais para capturar padrões.

Bias

Erro causado por simplificação excessiva.

Variance

Sensibilidade excessiva às variações dos dados.

Validação cruzada

Avaliação em múltiplas divisões para estimar generalização.

Tuning de hiperparâmetros

Busca por melhores configurações não aprendidas diretamente pelo modelo.

Métodos:

- Grid search.
- Random search.
- Bayesian optimization.

Baseline

Modelo simples usado como referência mínima.

Curva de aprendizado

Ajuda a diagnosticar se o problema é falta de dados, excesso de complexidade ou modelo fraco.

Verdade prática de mercado

Para muitos problemas tabulares reais, como fraude, churn, risco e previsão, gradient boosting frequentemente vence redes neurais.

Deep Learning brilha principalmente em texto, imagem, áudio e grandes volumes de dados não estruturados.

Mais profundo não significa automaticamente melhor.

Ferramentas

- Scikit-learn.
- XGBoost.
- LightGBM.

- CatBoost.
- MLflow.

Erros comuns

- Olhar apenas acurácia.
- Ignorar desbalanceamento de classes.
- Não usar validação cruzada.
- Comparar modelos de forma injusta.
- Não criar baseline.
- Usar Deep Learning sem necessidade.
- Não detectar overfitting.
- Não separar teste corretamente.

Critério de domínio

Você domina Machine Learning quando consegue escolher algoritmo e métrica corretos para um problema, justificar trade-offs, detectar overfitting, comparar modelos e criar pipeline com validação cruzada e tuning.

14. Deep Learning

Deep Learning usa redes neurais com muitas camadas para aprender representações complexas. É a base de visão computacional moderna, NLP avançado, reconhecimento de voz, IA Generativa e LLMs.

Perceptron

Unidade básica de decisão linear.

Combina entradas, pesos e bias, depois aplica uma função de ativação.

MLP

Rede feedforward com múltiplas camadas densas.

Usada para dados tabulares simples, experimentos iniciais e fundamentos.

Funções de ativação

- ReLU.
- Sigmoid.
- Tanh.
- GELU.

Sem não linearidade, várias camadas lineares colapsariam em uma única transformação linear.

Backpropagation

Algoritmo que calcula gradientes propagando erro de trás para frente, usando a regra da cadeia.

Otimizadores

- SGD.
- Adam.
- AdamW.
- RMSProp.

CNN

Redes convolucionais para imagens e padrões espaciais.

Aplicações:

- Classificação de imagem.

- Detecção de objetos.
- OCR.
- Inspeção de qualidade.
- Visão computacional industrial.

RNN, LSTM e GRU

Arquiteturas para sequências.

- RNNs processam dados ordenados.
- LSTM e GRU ajudam a lidar com dependências longas e gradiente que desaparece.
- Hoje, Transformers substituíram grande parte das RNNs em NLP.

Autoencoders

Redes que comprimem e reconstroem dados.

Aplicações:

- Redução de dimensionalidade.
- Detecção de anomalias.
- Denoising.
- Aprendizado de representações.

Transformers

Arquitetura baseada em self-attention.

Cada token pode prestar atenção nos demais tokens em paralelo. Isso permitiu grande avanço em NLP e IA Generativa.

Transformers são a base de BERT, GPT e da maioria dos LLMs modernos.

Embeddings

Representações vetoriais densas de dados.

Texto, imagem, áudio e itens podem ser representados como vetores.

A ideia central é: proximidade geométrica representa similaridade semântica.

Attention

Mecanismo que pondera quais partes da entrada são mais relevantes para cada elemento.

É fundamental em Transformers, busca semântica e LLMs.

Transfer learning

Reaproveitamento de modelo pré-treinado em uma nova tarefa.

Fine-tuning

Adaptação de modelo pré-treinado para domínio ou tarefa específica.

Regularização

Técnicas para reduzir overfitting:

- Dropout.
- Batch normalization.
- Weight decay.
- Data augmentation.
- Early stopping.

PyTorch vs TensorFlow/Keras

PyTorch costuma ser mais usado em pesquisa, prototipação e projetos customizados.

TensorFlow/Keras é forte em ambientes produtivos, mobile, serving e fluxos padronizados.

A recomendação prática é escolher um como principal, geralmente PyTorch, e conhecer o outro.

Erros comuns

- Treinar rede neural para problema tabular simples sem necessidade.
- Ignorar normalização.
- Não usar validação.
- Confundir loss com métrica de negócio.
- Não controlar seed.
- Não versionar pesos.
- Treinar do zero quando transfer learning resolveria melhor.
- Subestimar custo de GPU.

Critério de domínio

Você domina Deep Learning quando consegue implementar uma MLP, uma CNN simples, usar embeddings, fazer fine-tuning de modelo pré-treinado, explicar forward/backward pass e diagnosticar treino que não converge.

15. Processamento de Linguagem Natural — NLP

NLP é a área que trata texto, linguagem e significado computacional. É a ponte entre Machine Learning clássico e LLMs.

Entender NLP ajuda a compreender busca semântica, embeddings, RAG e modelos de linguagem.

- NLP clássico
- Tokenização

Quebra texto em unidades menores, como palavras, subpalavras ou tokens.

Stemming

Reduz palavras à raiz aproximada.

Lematização

Reduz palavras à forma canônica.

Stopwords

Palavras muito frequentes que podem ser removidas em algumas abordagens clássicas.

- Representações esparsas
- Bag of Words

Representa texto pela frequência de palavras.

TF-IDF

Pondera palavras pela frequência no documento e raridade no corpus.

Ainda é útil como baseline e em busca lexical, como BM25.

Embeddings clássicos

- Word2Vec.
- GloVe.
- FastText.

Esses modelos transformam palavras em vetores, permitindo capturar relações semânticas.

Modelos contextuais

- BERT.
- RoBERTa.
- DistilBERT.

A diferença central é que o significado da palavra depende do contexto.

Exemplo: “banco” em “banco financeiro” e “banco da praça” deve ter representações diferentes.

Tarefas de NLP

- Classificação de texto.
- Análise de sentimento.
- NER — Named Entity Recognition.
- Sumarização.
- Tradução.
- Question answering.
- Extração de entidades.
- Busca semântica.
- Roteamento de tickets.
- Leitura de contratos.
- Classificação de e-mails.
- Extração de dados de notas fiscais.

NLP moderno

- Transformers.
- LLMs.
- RAG.
- Busca semântica.
- Embeddings contextuais.
- Agentes conversacionais.

Aplicações reais

- Análise de chamados.
- Classificação de e-mails.
- Leitura de contratos.
- Extração de entidades de notas fiscais.
- Busca inteligente em base documental.
- Atendimento automatizado.
- Sumarização de relatórios.
- Assistentes internos.

Erros comuns

- Usar TF-IDF para problema que exige semântica profunda.
- Usar LLM para tarefa simples que um classificador resolveria melhor.
- Não tratar idioma.
- Não tratar domínio.
- Avaliar sumarização apenas subjetivamente.
- Não criar conjunto de avaliação.
- Não comparar com baseline lexical.

Critério de domínio

Você domina NLP quando consegue construir um classificador de texto, uma busca semântica com embeddings e um pipeline de extração de entidades ou question answering.

16. IA Generativa e LLMs

IA Generativa cria novos conteúdos a partir de padrões aprendidos. LLMs são modelos grandes de linguagem treinados para prever e gerar texto, podendo responder perguntas, resumir, traduzir, classificar, gerar código e interagir com ferramentas.

- Conceitos obrigatórios
- Modelos generativos

Aprendem a distribuição dos dados e geram novas amostras.

LLMs

Modelos de linguagem de grande escala treinados em grandes volumes de dados.

Tokens

Unidades processadas pelo modelo.

Podem representar palavras, subpalavras, símbolos ou partes de palavras.

Context window

Limite de tokens que o modelo consegue considerar na entrada e saída.

Embeddings

Representações vetoriais de texto, imagem ou outros dados.

Pre-training

Treinamento amplo em grandes corpora.

Fine-tuning

Adaptação supervisionada para tarefa ou domínio específico.

Instruction tuning

Treinamento para seguir instruções.

RLHF

Alinhamento usando feedback humano.

Prompt engineering

Desenho de instruções, contexto, formato e restrições.

System prompt

Instrução de alto nível que define comportamento do modelo.

Few-shot prompting

Uso de exemplos no prompt.

Raciocínio estruturado em produção

Em produção, prefira etapas auditáveis, checklists, validação externa e decomposição verificável, em vez de depender de raciocínio oculto do modelo.

RAG — Retrieval-Augmented Generation

RAG combina busca/retrieval com geração.

Fluxo típico:

Documento → Chunking → Embeddings → Banco vetorial → Consulta → Recuperação de trechos → Prompt com contexto → Resposta do LLM → Validação.

RAG é útil quando o modelo precisa responder com base em documentos específicos, dados privados ou conhecimento atualizado.

Componentes de um RAG

- Chunking.
- Embeddings.
- Vector database.
- Busca semântica.
- Busca híbrida.
- Reranking.
- Prompt com contexto.
- Geração.
- Citações.
- Validação.
- Guardrails.
- Observabilidade.

Bancos vetoriais

- Pinecone.
- Weaviate.
- Qdrant.
- Milvus.
- Chroma.
- pgvector.

Agentes

Agentes são sistemas que usam LLMs para decidir ações, chamar ferramentas, consultar APIs, executar passos e completar tarefas.

Componentes:

- Modelo.
- Prompt.
- Memória.
- Ferramentas.
- Planejamento.
- Execução.
- Observação.
- Validação.
- Limites de segurança.

Tool use e function calling

Permitem que LLMs chamem funções externas, APIs, bancos de dados, calculadoras, sistemas internos ou ferramentas específicas.

Guardrails

Mecanismos para limitar comportamento indesejado.

Exemplos:

- Validação de entrada.
- Validação de saída.
- Filtros de conteúdo.
- Restrições de formato.
- Checagem de fontes.
- Bloqueio de dados sensíveis.
- Detecção de prompt injection.

Avaliação de LLMs

Avaliação deve ir além de “a resposta parece boa”.

Critérios:

- Acurácia factual.
- Relevância.
- Completude.
- Aderência ao contexto.
- Taxa de alucinação.
- Custo por resposta.
- Latência.
- Segurança.
- Consistência.
- Qualidade de citação.
- Robustez contra prompt injection.

Erros comuns

- Tratar LLM como caixa mágica.
- Não avaliar respostas.
- Ignorar custo por token.
- Ignorar latência.
- Não usar RAG quando precisa de conhecimento específico.
- Usar fine-tuning quando RAG bastaria.
- Não testar prompt injection.
- Não criar dataset de avaliação.
- Não monitorar produção.

Critério de domínio

Você domina IA Generativa e LLMs quando consegue construir e avaliar objetivamente um sistema generativo com prompts estruturados, RAG, embeddings, banco vetorial, controle de custo, mitigação de alucinação e integração com ferramentas.

17. Engenharia de aplicações com IA

Esta é uma das áreas mais importantes para quem vem de backend. Construir aplicações de IA confiáveis é, em grande parte, aplicar boa engenharia de sistemas distribuídos a componentes probabilísticos, caros e sensíveis.

Arquitetura típica de uma aplicação com LLM

- API REST ou gRPC.
- Camada de autenticação.
- Camada de orquestração.
- Pipeline de recuperação.
- Banco vetorial.
- Busca híbrida.
- Reranking.
- Chamada ao modelo.
- Pós-processamento.
- Guardrails.
- Cache.
- Logs.
- Tracing.
- Métricas.
- Persistência.
- Monitoramento de custo.
- Provedores gerenciados vs modelos locais
- Provedores gerenciados

Exemplos:

- OpenAI.
- Anthropic.
- Google.
- AWS Bedrock.
- Azure OpenAI.

Vantagens:

- Menor operação.
- Alta qualidade.
- Escalabilidade gerenciada.
- Atualizações automáticas.

Desvantagens:

- Custo por token.
- Dependência externa.
- Questões de privacidade.
- Menor controle.

Modelos locais ou self-hosted

Exemplos:

- Hugging Face.
- Ollama.
- vLLM.
- Modelos open-source.

Vantagens:

- Mais controle.
- Privacidade.

- Possível redução de custo em escala.
- Customização.

Desvantagens:

- Exige infraestrutura.
- GPU.
- Operação.
- Monitoramento.
- Atualização.
- Equipe técnica mais madura.
- Padrões de produção
- Performance e UX
- Streaming de respostas token a token.
- Cache semântico.
- Cache de respostas.
- Chunking otimizado.
- Indexação eficiente.
- Pré-processamento assíncrono.
- Filas para tarefas pesadas.

Confiabilidade e custo

- Rate limits.
- Retries com backoff.
- Fallback de modelo.
- Timeouts.
- Circuit breaker.
- Controle de tokens.
- Controle de custo por usuário.
- Observabilidade de latência e erro.

Observabilidade

- Logs estruturados.
- Tracing distribuído.
- Métricas de tokens.
- Métricas de custo.
- Métricas de latência.
- Taxa de erro.
- Taxa de fallback.
- Taxa de alucinação.
- Feedback do usuário.
- Dashboards.

Segurança e privacidade

Nunca envie dados sensíveis para provedores externos sem política clara.

Boas práticas:

- Isolar segredos.
- Validar entrada.
- Validar saída.
- Reduzir dados no prompt.
- Mascaram informações sensíveis.
- Aplicar controle de acesso.

- Auditar chamadas.
- Registrar decisões.
- Testar prompt injection.
- Aplicar governança.

Exemplo de arquitetura para portfólio

- Frontend: Next.js.
- Backend: Spring Boot ou Python FastAPI.
- Banco relacional: PostgreSQL.
- Busca vetorial: pgvector ou Qdrant.
- Cache: Redis.
- Orquestração: LangChain, LlamaIndex ou código próprio.
- Modelos: API externa ou modelo local.
- Deploy: Docker.
- Observabilidade: logs, métricas e tracing.
- MLOps: MLflow para experimentos.
- Segurança: autenticação, autorização, validação e auditoria.

Critério de domínio

Você domina engenharia de aplicações com IA quando consegue projetar e implementar um serviço de IA com streaming, cache, observabilidade de custo, fallback, guardrails, segurança e prontidão para tráfego real.

18. MLOps e ciclo de vida de modelos

MLOps é DevOps aplicado a Machine Learning, com complexidades extras: além de código, você versiona dados, modelos, experimentos, features e métricas. O sistema também pode degradar sem mudança de código, porque o mundo muda.

Ciclo de vida de modelos

- Coleta de dados.
- Validação de dados.
- Preparação.
- Feature engineering.
- Treino.
- Avaliação.
- Registro do modelo.
- Deploy.
- Serving.
- Monitoramento.
- Detecção de drift.
- Retreino.
- Auditoria.
- Governança.

Experiment tracking

Registro de:

- Parâmetros.
- Hiperparâmetros.
- Métricas.
- Artefatos.

- Datasets.
- Versões de código.
- Versões de modelo.
- Resultados.

Ferramenta principal:

MLflow.

- Versionamento
 - Código
- Git.

Dados

- DVC.
- LakeFS.
- Controle por snapshots.
- Data versioning.

Modelos

- Model registry.
- MLflow Model Registry.
- Versionamento de pesos.
- Tags.
- Staging.
- Production.
- Archived.

Feature Store

Armazena e serve features consistentes para treino e produção.

Ferramentas:

- Feast.
- Tecton.
- Soluções internas.

Deploy e serving

Formas comuns:

- Batch prediction.
- Online inference.
- Real-time API.
- Streaming inference.
- Edge deployment.
- Serverless.
- Kubernetes.

Ferramentas:

- Docker.
- Kubernetes.
- FastAPI.
- BentoML.
- KServe.
- Seldon.
- TorchServe.
- TensorFlow Serving.

- vLLM para serving de LLM.

Monitoramento

Métricas técnicas:

- Latência.
- Throughput.
- Taxa de erro.
- Uso de CPU/GPU.
- Memória.
- Custo.

Métricas de ML:

- Acurácia.
- Precision.
- Recall.
- F1.
- AUC.
- Erro de regressão.
- Distribuição de features.
- Drift de dados.
- Drift de conceito.
- Qualidade de predição.

Métricas de LLM:

- Custo por chamada.
- Tokens de entrada.
- Tokens de saída.
- Latência.
- Taxa de alucinação.
- Taxa de rejeição.
- Feedback.
- Qualidade de recuperação.
- Precisão de citação.
- Drift
- Data drift

Mudança na distribuição dos dados de entrada.

Concept drift

Mudança na relação entre entrada e saída.

Model drift

Degradação do desempenho do modelo ao longo do tempo.

Retreino

Pode ser:

- Manual.
- Agendado.
- Disparado por métrica.
- Disparado por drift.
- Disparado por queda de performance.

CI/CD para ML

- Testes de código.
- Testes de dados.
- Validação de schema.
- Validação de distribuição.
- Treino automatizado.
- Avaliação automatizada.
- Deploy controlado.
- Rollback.
- Canary release.
- Shadow deployment.
- A/B testing.

Erros comuns

- Achar que deploy é o fim.
- Não versionar dados.
- Não versionar modelo.
- Não registrar experimentos.
- Não monitorar drift.
- Não automatizar validação.
- Não criar rollback.
- Não ter registry.
- Não auditar decisões.

Critério de domínio

Você domina MLOps quando consegue criar pipeline reproduzível de treino, tracking, registry, deploy, monitoramento, drift e retreino com controle de versões e governança.

19. Cloud e infraestrutura para IA

Cloud e infraestrutura são essenciais porque IA pode exigir armazenamento, processamento distribuído, GPUs, serving escalável, filas, bancos vetoriais e monitoramento.

Componentes de infraestrutura

- GPUs.
- TPUs.
- CPUs.
- Object storage.
- Data lake.
- Bancos relacionais.
- Bancos NoSQL.
- Bancos vetoriais.
- Filas.
- Kubernetes.
- Docker.
- APIs.
- Gateways.
- Observabilidade.
- Cache.
- CDN.

- Serviços gerenciados.

AWS

- SageMaker AI.
- Bedrock.
- S3.
- ECR.
- ECS.
- EKS.
- Lambda.
- CloudWatch.
- IAM.
- OpenSearch.
- RDS.
- ElastiCache.

Azure

- Azure AI Foundry.
- Azure OpenAI.
- Azure Machine Learning.
- Blob Storage.
- AKS.
- Functions.
- Application Insights.
- Key Vault.
- Cosmos DB.
- PostgreSQL.

Google Cloud

- Vertex AI.
- Cloud Storage.
- GKE.
- Cloud Run.
- BigQuery.
- Pub/Sub.
- Cloud Monitoring.

Serving de LLMs

- vLLM.
- Ollama.
- TGI — Text Generation Inference.
- TensorRT-LLM.
- KServe.
- Ray Serve.

Estratégias de deploy

- API gerenciada.
- Modelo local.
- Modelo self-hosted em GPU.
- Batch offline.

- Online inference.
- Serverless.
- Kubernetes.
- Edge.

Trade-offs

- Custo.
- Latência.
- Privacidade.
- Controle.
- Escalabilidade.
- Complexidade operacional.
- Disponibilidade.
- Vendor lock-in.

Erros comuns

- Usar GPU sem necessidade.
- Não calcular custo por requisição.
- Não monitorar tokens.
- Não definir SLOs.
- Não proteger chaves.
- Não planejar escalabilidade.
- Não ter fallback.
- Não estimar latência.

Critério de domínio

Você domina cloud para IA quando consegue escolher entre API externa, modelo local e serviço gerenciado, justificando custo, latência, segurança, privacidade e operação.

20. Segurança, ética e governança em IA

Segurança e governança não são temas opcionais. Sistemas de IA podem expor dados sensíveis, gerar decisões incorretas, reproduzir vieses, sofrer ataques e criar riscos regulatórios.

- Segurança em LLMs
- Prompt injection

Ataque em que o usuário tenta manipular o modelo para ignorar instruções ou revelar dados.

Data exfiltration

Vazamento de dados por meio de respostas do modelo.

Insecure output handling

Uso inseguro da saída do modelo em comandos, SQL, HTML ou APIs.

Model denial of service

Uso abusivo para gerar custo, lentidão ou indisponibilidade.

Supply chain

Risco em modelos, datasets, dependências e pipelines externos.

OWASP LLM Top 10

Referência para riscos de aplicações com LLMs.

Tópicos importantes:

- Prompt injection.
- Insecure output handling.
- Training data poisoning.
- Model denial of service.
- Supply chain vulnerabilities.
- Sensitive information disclosure.
- Insecure plugin design.
- Excessive agency.
- Overreliance.
- Model theft.

Privacidade

- LGPD.
- GDPR.
- Minimização de dados.
- Anonimização.
- Pseudonimização.
- Controle de acesso.
- Retenção de dados.
- Consentimento.
- Auditoria.

Ética

- Viés.
- Fairness.
- Transparência.
- Explicabilidade.
- Responsabilidade.
- Impacto social.
- Uso adequado.
- Limites de automação.

Governança

- Políticas.
- Auditoria.
- Model cards.
- Data cards.
- Rastreabilidade.
- Registro de decisões.
- Inventário de modelos.
- Aprovação de uso.
- Monitoramento contínuo.
- Plano de resposta a incidentes.

NIST AI RMF

Framework para gerenciar riscos de IA em indivíduos, organizações e sociedade.

Ajuda a estruturar práticas de governança, avaliação, documentação e mitigação de riscos.

Erros comuns

- Tratar segurança como etapa final.
- Enviar dados sensíveis para APIs externas sem controle.
- Não testar prompt injection.
- Não documentar datasets.
- Não auditar decisões.
- Não monitorar viés.
- Não ter política de retenção.
- Não explicar limitações do modelo.
- Confiar cegamente na resposta do LLM.

Critério de domínio

Você domina segurança e governança quando consegue projetar uma solução de IA com controle de acesso, privacidade, auditoria, mitigação de prompt injection, rastreabilidade e documentação de riscos.

21. Projetos práticos por nível

Os projetos foram padronizados como fichas técnicas para facilitar execução, documentação e uso em portfólio.

Nível 1 — Fundamentos

Campo	Descrição
Nome do projeto	Projeto 1.1 - Análise exploratória de dados e previsão de preços
Nível técnico	Nível 1 — Fundamentos
Objetivo	Trabalhar com um dataset tabular, fazer limpeza, análise exploratória, visualização, feature engineering simples e treinar um modelo inicial de regressão.
Conteúdos aplicados	Pandas. Matplotlib. Estatística básica. Regressão linear. Métricas de regressão. Train/test split. README técnico.
Entregável	Notebook limpo, relatório de análise, modelo baseline e explicação das métricas.
Critérios de qualidade	Entregável documentado, reproduzível, versionado em Git, com README técnico, validação objetiva e evidências de execução.
Possíveis melhorias	Adicionar testes, observabilidade, deploy, segurança, avaliação contínua e integração com sistemas reais conforme o nível do projeto.
Competências desenvolvidas	Pandas. Matplotlib. Estatística básica. Regressão linear. Métricas de regressão. Train/test split. README técnico.

Campo	Descrição
Nome do projeto	Projeto 1.2 - Classificador de spam ou Titanic
Nível técnico	Nível 1 — Fundamentos
Objetivo	Criar um primeiro modelo de classificação.
Conteúdos aplicados	Classificação. Regressão logística. Naive Bayes. Decision Tree. Matriz de confusão. Precision. Recall. F1. Validação simples.
Entregável	Modelo treinado, comparação de métricas e explicação dos erros.
Critérios de qualidade	Entregável documentado, reproduzível, versionado em Git, com README técnico, validação objetiva e evidências de execução.
Possíveis melhorias	Adicionar testes, observabilidade, deploy, segurança, avaliação contínua e integração com sistemas reais conforme o nível do projeto.
Competências desenvolvidas	Classificação. Regressão logística. Naive Bayes. Decision Tree. Matriz de confusão. Precision. Recall. F1. Validação simples.

Nível 2 — Intermediário

Campo	Descrição
Nome do projeto	Projeto 2.1 - Previsão de churn com gradient boosting
Nível técnico	Nível 2 — Intermediário
Objetivo	Construir modelo tabular mais robusto para prever cancelamento de clientes.
Conteúdos aplicados	Feature engineering. XGBoost. LightGBM. CatBoost. Cross-validation. Tuning. AUC. Recall. F1. Interpretação de features.
Entregável	Pipeline validado, tabela de comparação de modelos e análise dos principais fatores de churn.
Critérios de qualidade	Entregável documentado, reproduzível, versionado em Git, com README técnico, validação objetiva e evidências de execução.
Possíveis melhorias	Adicionar testes, observabilidade, deploy, segurança, avaliação contínua e integração com sistemas reais conforme o nível do projeto.
Competências desenvolvidas	Feature engineering. XGBoost. LightGBM. CatBoost. Cross-validation. Tuning. AUC. Recall. F1. Interpretação de features.

Campo	Descrição
Nome do projeto	Projeto 2.2 - API de ML servindo um modelo
Nível técnico	Nível 2 — Intermediário
Objetivo	Expor um modelo treinado via API.
Conteúdos aplicados	FastAPI ou Spring Boot consumindo modelo. Serialização do modelo. Docker. Validação de entrada. Logs. Testes. Documentação OpenAPI.
Entregável	API funcional com endpoint de predição, Dockerfile, README e exemplos de request/response.
Critérios de qualidade	Entregável documentado, reproduzível, versionado em Git, com README técnico, validação objetiva e evidências de execução.
Possíveis melhorias	Adicionar testes, observabilidade, deploy, segurança, avaliação contínua e integração com sistemas reais conforme o nível do projeto.
Competências desenvolvidas	FastAPI ou Spring Boot consumindo modelo. Serialização do modelo. Docker. Validação de entrada. Logs. Testes. Documentação OpenAPI.

Campo	Descrição
Nome do projeto	Projeto 2.3 - Segmentação de clientes
Nível técnico	Nível 2 — Intermediário

Campo	Descrição
Objetivo	Aplicar aprendizado não supervisionado para encontrar grupos de clientes.
Conteúdos aplicados	K-Means. DBSCAN. PCA. Normalização. Visualização. Análise qualitativa.
Entregável	Clusters interpretados, visualizações e recomendações de negócio.
Critérios de qualidade	Entregável documentado, reproduzível, versionado em Git, com README técnico, validação objetiva e evidências de execução.
Possíveis melhorias	Adicionar testes, observabilidade, deploy, segurança, avaliação contínua e integração com sistemas reais conforme o nível do projeto.
Competências desenvolvidas	K-Means. DBSCAN. PCA. Normalização. Visualização. Análise qualitativa.

Nível 3 — Avançado

Campo	Descrição
Nome do projeto	Projeto 3.1 - Classificador de imagens com CNN e transfer learning
Nível técnico	Nível 3 — Avançado
Objetivo	Construir um classificador de imagens usando modelo pré-treinado.
Conteúdos aplicados	PyTorch ou TensorFlow/Keras. CNN. Transfer learning. Data augmentation. Fine-tuning. Métricas. Matriz de confusão.
Entregável	Modelo treinado, avaliação, notebook/script e explicação de erros.
Critérios de qualidade	Entregável documentado, reproduzível, versionado em Git, com README técnico, validação objetiva e evidências de execução.
Possíveis melhorias	Adicionar testes, observabilidade, deploy, segurança, avaliação contínua e integração com sistemas reais conforme o nível do projeto.
Competências desenvolvidas	PyTorch ou TensorFlow/Keras. CNN. Transfer learning. Data augmentation. Fine-tuning. Métricas. Matriz de confusão.

Campo	Descrição
Nome do projeto	Projeto 3.2 - NLP com BERT
Nível técnico	Nível 3 — Avançado
Objetivo	Criar um classificador de texto ou pipeline de question answering com modelo Transformer.
Conteúdos aplicados	Hugging Face. Tokenização. BERT. Embeddings contextuais. Fine-tuning. Avaliação.

Campo	Descrição
Entregável	Pipeline de NLP com dataset real, métricas e exemplos de inferência.
Critérios de qualidade	Entregável documentado, reproduzível, versionado em Git, com README técnico, validação objetiva e evidências de execução.
Possíveis melhorias	Adicionar testes, observabilidade, deploy, segurança, avaliação contínua e integração com sistemas reais conforme o nível do projeto.
Competências desenvolvidas	Hugging Face. Tokenização. BERT. Embeddings contextuais. Fine-tuning. Avaliação.

Campo	Descrição
Nome do projeto	Projeto 3.3 - RAG sobre documentos próprios
Nível técnico	Nível 3 — Avançado
Objetivo	Construir sistema de perguntas e respostas sobre documentos.
Conteúdos aplicados	Chunking. Embeddings. Vector database. Busca semântica. RAG. Prompt com contexto. Citações. Validação de resposta.
Entregável	Chatbot documental com API ou interface simples, documentação e avaliação.
Critérios de qualidade	Entregável documentado, reproduzível, versionado em Git, com README técnico, validação objetiva e evidências de execução.
Possíveis melhorias	Adicionar testes, observabilidade, deploy, segurança, avaliação contínua e integração com sistemas reais conforme o nível do projeto.
Competências desenvolvidas	Chunking. Embeddings. Vector database. Busca semântica. RAG. Prompt com contexto. Citações. Validação de resposta.

Campo	Descrição
Nome do projeto	Projeto 3.4 - Fine-tuning com LoRA/QLoRA
Nível técnico	Nível 3 — Avançado
Objetivo	Adaptar modelo de linguagem a uma tarefa ou domínio específico com técnica eficiente.
Conteúdos aplicados	Fine-tuning. LoRA. QLoRA. Datasets supervisionados. Avaliação. Controle de custo.
Entregável	Modelo ajustado, comparação antes/depois e relatório de limitações.
Critérios de qualidade	Entregável documentado, reproduzível, versionado em Git, com README técnico, validação objetiva e evidências de execução.
Possíveis melhorias	Adicionar testes, observabilidade, deploy, segurança, avaliação contínua e integração com sistemas reais conforme o nível do projeto.

Campo	Descrição
Competências desenvolvidas	Fine-tuning. LoRA. QLoRA. Datasets supervisionados. Avaliação. Controle de custo.

Campo	Descrição
Nome do projeto	Projeto 3.5 - Agente com function calling
Nível técnico	Nível 3 — Avançado
Objetivo	Criar agente que consulta ferramentas externas.
Conteúdos aplicados	LLM. Function calling. Tool use. APIs. Validação. Guardrails. Logs.
Entregável	Agente que chama funções reais, registra passos e retorna resposta validada.
Critérios de qualidade	Entregável documentado, reproduzível, versionado em Git, com README técnico, validação objetiva e evidências de execução.
Possíveis melhorias	Adicionar testes, observabilidade, deploy, segurança, avaliação contínua e integração com sistemas reais conforme o nível do projeto.
Competências desenvolvidas	LLM. Function calling. Tool use. APIs. Validação. Guardrails. Logs.

Nível 4 — Profissional/Sênior

Campo	Descrição
Nome do projeto	Projeto 4.1 - Plataforma RAG ponta a ponta em produção
Nível técnico	Nível 4 — Profissional/Sênior
Objetivo	Criar uma solução RAG completa, com arquitetura profissional.
Componentes	Frontend. Backend. Banco relacional. Banco vetorial. Cache. Autenticação. Upload de documentos. Pipeline de indexação. Busca semântica. Reranking. LLM. Streaming. Observabilidade. Controle de custo. Guardrails. Deploy.
Stack sugerida	Next.js. Spring Boot ou FastAPI. PostgreSQL + pgvector ou Qdrant. Redis. Docker. OpenAI/Anthropic/Gemini ou modelo local. MLflow ou logging customizado.
Entregável	Aplicação completa, documentada, segura, observável e publicável em portfólio.
CrITÉRIOS de qualidade	Entregável documentado, reproduzível, versionado em Git, com README técnico, validação objetiva e evidências de execução.
PossÍveis melhorias	Adicionar testes, observabilidade, deploy, segurança, avaliação contínua e integração com sistemas reais conforme o nível do projeto.
Competências desenvolvidas	Frontend. Backend. Banco relacional. Banco vetorial. Cache. Autenticação. Upload de documentos. Pipeline de indexação. Busca semântica. Reranking. LLM. Streaming. Observabilidade. Controle de custo. Guardrails. Deploy.

Campo	Descrição
Nome do projeto	Projeto 4.2 - Pipeline MLOps completo com retreino por drift
Nível técnico	Nível 4 — Profissional/Sênior
Objetivo	Construir pipeline profissional de ML com monitoramento e retreino.

Campo	Descrição
Componentes	Coleta de dados. Validação. Treino. Tracking. Registry. Deploy. Monitoramento. Detecção de drift. Retreino. CI/CD. Rollback.
Ferramentas	MLflow. DVC. Docker. Kubernetes. Evidently. Airflow ou Kubeflow. Cloud. GitHub Actions ou Azure DevOps.
Entregável	Pipeline automatizado com documentação arquitetural e evidência de monitoramento.
Critérios de qualidade	Entregável documentado, reproduzível, versionado em Git, com README técnico, validação objetiva e evidências de execução.
Possíveis melhorias	Adicionar testes, observabilidade, deploy, segurança, avaliação contínua e integração com sistemas reais conforme o nível do projeto.
Competências desenvolvidas	Coleta de dados. Validação. Treino. Tracking. Registry. Deploy. Monitoramento. Detecção de drift. Retreino. CI/CD. Rollback. MLflow. DVC. Docker. Kubernetes. Evidently. Airflow ou Kubeflow. Cloud. GitHub Actions ou Azure DevOps.

Campo	Descrição
Nome do projeto	Projeto 4.3 - Plataforma de IA corporativa
Nível técnico	Nível 4 — Profissional/Sênior
Objetivo	Projetar uma solução corporativa de IA com governança.
Componentes	Controle de usuários. Auditoria. Logs. Políticas de uso. Catálogo de modelos. Registro de prompts. Controle de custo. Avaliação. Segurança. LGPD. Rastreabilidade. Dashboards. APIs internas.
Entregável	Arquitetura de referência, protótipo funcional e documentação de decisões técnicas.
Critérios de qualidade	Entregável documentado, reproduzível, versionado em Git, com README técnico, validação objetiva e evidências de execução.

Campo	Descrição
Possíveis melhorias	Adicionar testes, observabilidade, deploy, segurança, avaliação contínua e integração com sistemas reais conforme o nível do projeto.
Competências desenvolvidas	Controle de usuários. Auditoria. Logs. Políticas de uso. Catálogo de modelos. Registro de prompts. Controle de custo. Avaliação. Segurança. LGPD. Rastreabilidade. Dashboards. APIs internas.

22. Grade de estudos do básico ao avançado

A grade de estudos foi convertida em tabelas por nível, mantendo o padrão do conteúdo original: definição, uso, pré-requisitos, prática, ferramentas, erros comuns e critério de domínio.

Nível 1 — Fundamentos

Tema	Definição e uso	Pré-requisitos e prática	Ferramentas	Erros comuns e domínio
Conceitos de IA	O que é: Base teórica sobre IA, ML, Deep Learning, GenAI e LLMs. Para que serve: Entender a área e evitar uso superficial de ferramentas.	Pré-requisitos: Lógica básica. Prática: Criar mapas conceituais e exemplos de problemas.	Artigos, cursos introdutórios, documentação oficial.	Erros comuns: Achar que IA é igual a ChatGPT. Domínio: Explicar IA, ML, DL, GenAI e LLM com exemplos.
Matemática básica	O que é: Vetores, matrizes, estatística, probabilidade e função de perda. Para que serve: Entender como modelos representam dados e reduzem erro.	Pré-requisitos: Álgebra escolar. Prática: Exercícios com NumPy.	NumPy, Matplotlib.	Erros comuns: Decorar fórmula sem aplicação. Domínio: Interpretar métricas e explicar erro.
Python básico	O que é: Linguagem principal para IA. Para que serve: Programar experimentos, pipelines e modelos.	Pré-requisitos: Lógica de programação. Prática: Scripts e notebooks.	Python, Jupyter, VS Code.	Erros comuns: Código solto e não reproduzível. Domínio: Scripts organizados.
Dados básicos	O que é: Limpeza, EDA e preparação de dataset. Para que serve: Preparar dados para modelagem.	Pré-requisitos: Python. Prática: EDA em CSV.	Pandas.	Erros comuns: Ignorar missing values. Domínio: Dataset limpo e documentado.

Nível 2 — Intermediário

Tema	Definição e uso	Pré-requisitos e prática	Ferramentas	Erros comuns e domínio
Machine Learning supervisionado	O que é: Modelos treinados com rótulo. Para que serve: Classificação e regressão.	Pré-requisitos: Dados e métricas. Prática: Treinar regressão e classificação.	Scikit-learn.	Erros comuns: Avaliar apenas com acurácia. Domínio: Comparar modelos corretamente.
Machine Learning não supervisionado	O que é: Modelos que descobrem padrões sem rótulo. Para que serve: Segmentação, agrupamento e redução de dimensionalidade.	Pré-requisitos: Álgebra, estatística e dados. Prática: K-Means, PCA e DBSCAN.	Scikit-learn.	Erros comuns: Interpretar cluster sem análise. Domínio: Explicar grupos encontrados.
Métricas e validação	O que é: Formas de medir desempenho e generalização. Para que serve: Evitar conclusões falsas.	Pré-requisitos: ML básico. Prática: Cross-validation, matriz de confusão, AUC, RMSE.	Scikit-learn.	Erros comuns: Data leakage. Domínio: Escolher métrica adequada ao problema.
Feature engineering	O que é: Criação de variáveis úteis. Para que serve: Melhorar desempenho e capturar sinal.	Pré-requisitos: Dados e domínio de negócio. Prática: Criar variáveis temporais, agregadas e categóricas.	Pandas, Scikit-learn.	Erros comuns: Criar feature com informação futura. Domínio: Justificar cada feature.
API de modelo	O que é: Serviço que expõe modelo para consumo. Para que serve: Integrar ML a sistemas reais.	Pré-requisitos: Backend e ML. Prática: Criar endpoint de predição.	FastAPI, Spring Boot, Docker.	Erros comuns: Não validar entrada. Domínio: Modelo acessível via API.

Nível 3 — Avançado

Tema	Definição e uso	Pré-requisitos e prática	Ferramentas	Erros comuns e domínio
Deep Learning	O que é: Redes neurais profundas. Para que serve: Resolver problemas complexos de imagem, texto, áudio e representação.	Pré-requisitos: ML e matemática. Prática: MLP e CNN.	PyTorch, TensorFlow/Keras.	Erros comuns: Usar rede neural sem necessidade. Domínio: Treinar e explicar rede simples.
NLP	O que é: Processamento computacional de linguagem. Para que serve: Classificar, extrair, resumir, buscar e responder com base em texto.	Pré-requisitos: ML e texto. Prática: Classificador de texto.	Scikit-learn, Hugging Face.	Erros comuns: Ignorar contexto e domínio. Domínio: Pipeline de texto funcional.
Transformers e embeddings	O que é: Arquiteturas e representações modernas para linguagem. Para que serve: Busca semântica, LLMs e RAG.	Pré-requisitos: NLP e Deep Learning. Prática: Gerar embeddings e calcular similaridade.	Hugging Face, sentence-transformers.	Erros comuns: Não avaliar qualidade da recuperação. Domínio: Busca semântica funcional.
RAG	O que é: Geração aumentada por recuperação. Para que serve: Responder com base em documentos específicos.	Pré-requisitos: Embeddings, banco vetorial e LLM. Prática: Chatbot documental.	LangChain, LlamaIndex, Qdrant, pgvector.	Erros comuns: Chunking ruim e ausência de avaliação. Domínio: RAG com respostas rastreáveis.
Agentes	O que é: LLMs usando ferramentas para executar tarefas. Para que serve: Automação inteligente.	Pré-requisitos: LLMs, APIs e segurança. Prática: Agente com function calling.	OpenAI API, LangChain, LlamaIndex.	Erros comuns: Excesso de autonomia sem guardrails. Domínio: Agente controlado e auditável.

Nível 4 — Profissional/Sênior

Tema	Definição e uso	Pré-requisitos e prática	Ferramentas	Erros comuns e domínio
Arquitetura de IA	O que é: Desenho ponta a ponta de sistemas inteligentes. Para que serve: Criar soluções escaláveis, seguras e sustentáveis.	Pré-requisitos: Backend, cloud, IA e MLOps. Prática: Desenhar arquitetura RAG ou ML em produção.	Diagramas, cloud, Docker, Kubernetes.	Erros comuns: Ignorar custo e operação. Domínio: Arquitetura justificável.
MLOps	O que é: Operação de modelos em produção. Para que serve: Garantir confiabilidade, versionamento e evolução.	Pré-requisitos: ML, DevOps e dados. Prática: Pipeline com MLflow e Docker.	MLflow, DVC, Docker, K8s.	Erros comuns: Não monitorar drift. Domínio: Pipeline reproduzível.
Segurança	O que é: Proteção de dados, prompts, modelos e aplicações. Para que serve: Reduzir riscos técnicos, legais e operacionais.	Pré-requisitos: Backend, IA e governança. Prática: Testar prompt injection e validação de saída.	OWASP LLM Top 10.	Erros comuns: Confiar cegamente no modelo. Domínio: Solução protegida.
Governança	O que é: Políticas, documentação, auditoria e controle. Para que serve: Compliance e rastreabilidade.	Pré-requisitos: Segurança e dados. Prática: Criar model cards e data cards.	NIST AI RMF, LGPD.	Erros comuns: Não documentar decisões. Domínio: Rastreabilidade.
Escalabilidade	O que é: Capacidade de atender produção real com custo e performance controlados. Para que serve: Evitar gargalos, latência alta e custo imprevisível.	Pré-requisitos: Cloud e arquitetura. Prática: vLLM, cache, filas e Kubernetes.	Kubernetes, Redis, vLLM.	Erros comuns: Custo imprevisível. Domínio: SLOs atendidos.

23. Cronograma de 3 meses — imersivo e aplicado

Semana	Conteúdo	Laboratório	Entregável
Semana 1	IA, ML, Deep Learning, GenAI, LLMs e tipos de problema.	Mapa mental da área.	Resumo técnico explicando IA, ML, DL, GenAI e LLM.
Semana 2	Python, NumPy e Pandas.	EDA em dataset simples.	Notebook limpo.
Semana 3	Estatística, métricas e visualização.	Análise exploratória.	Relatório de EDA.
Semana 4	Regressão e classificação.	Modelo simples.	Baseline.
Semana 5	Árvores, Random Forest e Gradient Boosting.	Comparar modelos.	Tabela de métricas.
Semana 6	Cross-validation, tuning e data leakage.	Pipeline com Scikit-learn.	Modelo validado.
Semana 7	Clustering, PCA e segmentação.	K-Means e PCA.	Análise de clusters.
Semana 8	Redes neurais, PyTorch ou Keras.	MLP simples.	Treino documentado.
Semana 9	CNN ou NLP introdutório.	Fine-tuning leve.	Modelo de Deep Learning.
Semana 10	Embeddings e busca semântica.	Vector search.	Buscador semântico.
Semana 11	RAG básico.	Chatbot documental.	API RAG.
Semana 12	Docker, MLflow, revisão e empacotamento.	Empacotar projeto.	Portfólio v1.

24. Cronograma de 6 meses — base sólida

Período	Conteúdo / Fase	Projetos	Resultado
Mês 1	Fundamentos, Python, matemática aplicada e EDA	Análise exploratória completa. Regressão simples. Classificação inicial.	Base técnica consolidada.
Mês 2	ML clássico, métricas, validação e tuning	Churn, fraude ou classificação tabular.	Modelos comparados corretamente.
Mês 3	Modelos tabulares avançados, clustering e pipelines	Gradient boosting com MLflow. Segmentação de clientes. API de modelo.	Pipeline intermediário de ML.
Mês 4	Deep Learning, PyTorch/TensorFlow, CNN e MLP	Classificador de imagem ou texto.	Base prática em Deep Learning.
Mês 5	NLP, embeddings, Transformers e RAG	Sistema RAG com documentos. Busca semântica. Classificador com BERT.	Aplicação moderna com LLM.
Mês 6	Produção, MLOps, observabilidade, segurança e deploy	Deploy com Docker. Tracking com MLflow. RAG com observabilidade. Projeto 4.1 versão inicial.	Solução publicável e próxima de produção.

25. Cronograma de 12 meses — formação avançada

Período	Conteúdo / Fase	Projetos	Resultado
Meses 1 a 2	Fundamentos		Nível 1 dominado.
Meses 3 a 5	Machine Learning		Nível 2 dominado.
Meses 6 a 8	Deep Learning, NLP e LLMs		Nível 3 dominado.
Meses 9 a 12	Produção e senioridade		Projetos profissionais de nível avançado.

26. Cronograma de 18 meses — nível profissional/sênior

Período	Conteúdo / Fase	Projetos	Resultado
Meses 1 a 3	Base sólida		Base teórica e prática.
Meses 4 a 6	Machine Learning intermediário		Projetos tabulares consistentes.
Meses 7 a 9	Deep Learning e NLP		Projetos de texto e imagem.
Meses 10 a 12	LLMs e IA Generativa		Aplicações inteligentes modernas.
Meses 13 a 15	MLOps e produção		Pipeline profissional.
Meses 16 a 18	Arquitetura, segurança e governança		

Projeto final de 18 meses

Plataforma de IA corporativa com:

- Autenticação.
- Upload de documentos.
- RAG.
- Busca semântica.
- LLM.
- Cache.
- Streaming.
- Logs.
- Métricas.
- Controle de custo.
- MLflow.
- Docker.
- Banco vetorial.
- Segurança.
- Governança.
- Documentação arquitetural.

27. Fontes oficiais e referências

Materiais de universidades

- Stanford CS229 — Machine Learning.
- Stanford CS224N — NLP com Deep Learning.
- MIT 6.S191 — Deep Learning.

Bibliotecas e frameworks

- PyTorch Documentation.
- TensorFlow/Keras Documentation.
- Scikit-learn Documentation.
- Pandas Documentation.
- NumPy Documentation.
- Hugging Face Transformers.

LLMs, RAG e bancos vetoriais

- LangChain Documentation.
- LlamaIndex Documentation.
- OpenAI Documentation.
- Anthropic Documentation.
- Qdrant Documentation.
- Pinecone Documentation.
- Weaviate Documentation.
- pgvector Documentation.
- vLLM Documentation.
- Ollama Documentation.

MLOps

- MLflow Documentation.
- DVC Documentation.
- Feast Documentation.
- Kubeflow Documentation.

Cloud

- AWS SageMaker AI.
- Azure AI Foundry.
- Google Vertex AI.

Segurança, privacidade e governança

- OWASP LLM Top 10.
- NIST AI Risk Management Framework.
- LGPD.
- GDPR.

Outras documentações técnicas

- Google Cloud — Conceitos de IA.

Livros recomendados

- Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow — Aurélien Géron.
- Deep Learning — Ian Goodfellow, Yoshua Bengio e Aaron Courville.

- Pattern Recognition and Machine Learning — Christopher Bishop.
- The Elements of Statistical Learning — Hastie, Tibshirani e Friedman.
- Designing Machine Learning Systems — Chip Huyen.
- Machine Learning Design Patterns — Valliappa Lakshmanan, Sara Robinson e Michael Munn.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Natural Language Processing with Transformers — Lewis Tunstall, Leandro von Werra e Thomas Wolf.
- AI Engineering — Chip Huyen.

28. Checklist de evolução técnica

Fundamentos

- ☐ Explico IA, ML, Deep Learning, GenAI e LLMs.
- ☐ Diferencio problemas de classificação, regressão, clustering, ranking, recomendação e geração.
- ☐ Sei quando usar IA e quando usar regra determinística.
- ☐ Entendo limitações, riscos e alucinações.
- ☐ Conheço aplicações reais de mercado.

Matemática

- ☐ Entendo vetores, matrizes e tensores.
- ☐ Explico produto escalar e similaridade.
- ☐ Entendo derivadas, gradientes e função de perda.
- ☐ Interpreto média, variância, desvio-padrão e distribuições.
- ☐ Entendo métricas de classificação e regressão.

Programação e dados

- ☐ Uso Python com organização.
- ☐ Manipulo dados com NumPy e Pandas.
- ☐ Faço EDA.
- ☐ Trato dados ausentes.
- ☐ Crio features.
- ☐ Evito data leakage.
- ☐ Crio pipelines reproduzíveis.
- ☐ Versiono código com Git.

Machine Learning

- ☐ Treino regressão, classificação e clustering.
- ☐ Comparo modelos com validação cruzada.
- ☐ Faço tuning de hiperparâmetros.
- ☐ Explico overfitting e underfitting.
- ☐ Explico bias e variance.
- ☐ Escolho métricas adequadas.
- ☐ Crio baseline.
- ☐ Uso Scikit-learn, XGBoost, LightGBM ou CatBoost.

Deep Learning e NLP

- ☐ Implemento uma MLP.
- ☐ Implemento ou uso uma CNN.
- ☐ Faço fine-tuning de modelo pré-treinado.
- ☐ Entendo embeddings.

- ☐ Entendo atenção.
- ☐ Explico Transformers em alto nível.
- ☐ Construo classificador de texto ou imagem.
- ☐ Uso Hugging Face.

LLMs e IA Generativa

- ☐ Entendo tokens, contexto, embeddings e Transformer.
- ☐ Crio prompts estruturados e avaliáveis.
- ☐ Construo RAG com documentos.
- ☐ Uso banco vetorial.
- ☐ Entendo agentes e tool use.
- ☐ Sei mitigar alucinações com recuperação e validação.
- ☐ Avalio respostas de LLMs.
- ☐ Considero custo e latência.

Engenharia e MLOps

- ☐ Exponho modelo via API.
- ☐ Uso MLflow para tracking.
- ☐ Versiono dados e modelos.
- ☐ Faço deploy com Docker.
- ☐ Entendo model registry.
- ☐ Monitoro latência, erro, custo e drift.
- ☐ Crio pipeline de retreino.
- ☐ Uso práticas de CI/CD para ML.

Cloud e infraestrutura

- ☐ Entendo uso de GPU e CPU.
- ☐ Conheço AWS SageMaker, Azure AI Foundry ou Vertex AI.
- ☐ Sei comparar API externa e modelo local.
- ☐ Uso cache para reduzir custo.
- ☐ Entendo Kubernetes para serving.
- ☐ Sei definir trade-offs entre custo, latência e privacidade.

Segurança e governança

- ☐ Entendo prompt injection.
- ☐ Aplico validação de entrada e saída.
- ☐ Protejo dados sensíveis.
- ☐ Conheço OWASP LLM Top 10.
- ☐ Conheço NIST AI RMF.
- ☐ Considero LGPD.
- ☐ Documento decisões técnicas.
- ☐ Crio rastreabilidade com logs e auditoria.

Sênior

- ☐ Desenho arquitetura de IA ponta a ponta.
- ☐ Tomo decisões entre modelo local, API externa e serviço gerenciado.
- ☐ Defino trade-offs entre custo, latência, privacidade e controle.
- ☐ Aplico segurança, LGPD, auditoria e governança.
- ☐ Crio soluções escaláveis, testáveis e observáveis.
- ☐ Documento arquitetura e decisões.

- ☐ Consigo operar IA em produção.
- ☐ Consigo orientar outros desenvolvedores na trilha.

29. Resultado esperado ao final da trilha

Ao final da trilha completa, o estudante deve ser capaz de entender IA de forma conceitual e técnica, sem depender apenas de ferramentas prontas.

Deve conseguir trabalhar com dados reais, tratando qualidade, preparação, features, governança, privacidade e versionamento.

Deve conseguir construir modelos de Machine Learning clássico, avaliando corretamente métricas, erros, trade-offs e limitações.

Deve conseguir desenvolver modelos de Deep Learning e NLP, usando PyTorch, TensorFlow/Keras, Hugging Face, embeddings, Transformers e fine-tuning.

Deve conseguir criar aplicações com LLMs, incluindo prompts estruturados, RAG, embeddings, vector databases, agentes, tool use e guardrails.

Deve conseguir projetar backends inteligentes com APIs, streaming, cache, logs, métricas, controle de custo, autenticação e segurança.

Deve conseguir aplicar MLOps, com tracking, versionamento, registry, deploy, monitoramento, drift, retreinamento e CI/CD.

Deve conseguir arquitetar soluções profissionais considerando escalabilidade, privacidade, LGPD, OWASP, NIST, cloud, observabilidade e governança.

Deve conseguir montar um portfólio técnico forte, com projetos progressivos e documentados.

O objetivo final não é formar alguém que apenas “usa IA”, mas alguém capaz de entender, construir, integrar, operar, proteger e governar sistemas inteligentes em produção.

30. Conclusão editorial

A trilha consolidada desenvolve competência completa em Inteligência Artificial: fundamentos, matemática, programação, dados, Machine Learning, Deep Learning, NLP, LLMs, IA Generativa, RAG, agentes, engenharia de aplicações, MLOps, cloud, segurança, ética, governança e arquitetura.

A progressão correta é sair do uso superficial de ferramentas e avançar para uma visão de engenharia: entender o problema, preparar dados, escolher abordagem, avaliar corretamente, integrar com sistemas reais, operar em produção, monitorar, proteger, reduzir custo e documentar decisões.

Essa é a diferença entre apenas consumir IA e atuar tecnicamente com IA em nível profissional/sênior.